Grand Manuel Linux — édition publique anonymisée

Extrait pédagogique V2 — novembre 2026

Objet : montrer comment, en moins d'un an d'apprentissage assisté par IA, des questions techniques quotidiennes deviennent des procédures comprises, testées et réutilisables.

> **Cette édition n'est pas une exportation brute du manuel privé.** Les noms de machines, domaines, comptes, chemins privés, adresses réseau, configurations d'accès à distance, secrets et identifiants de stockage en ligne ont été retirés. Les exemples sont génériques et ne reproduisent pas un réseau domestique réel.

1. Philosophie : apprendre, maîtriser, transmettre, construire

Le but n'est pas d'obtenir une suite de commandes à copier. Le but est de comprendre ce que l'on modifie, pourquoi on le fait, comment contrôler le résultat et comment revenir en arrière. Linux est utilisé comme un terrain d'apprentissage concret et un environnement que l'on peut s'approprier progressivement.

Règle d'or : vérifier avant d'exécuter. Toujours identifier le système cible, la version, l'état courant, les données à préserver et les conséquences possibles. Une bonne procédure distingue la commande informative de la commande qui modifie le système.

2. De la question au savoir vérifié

La fiche type du manuel comprend :

1. **Contexte et symptôme** — le problème observable, sans interprétation hâtive.
2. **Hypothèses** — plusieurs causes possibles, classées par coût et risque.
3. **Contrôles non destructifs** — état des services, journaux, versions, ressources.
4. **Intervention ciblée** — une modification à la fois, après sauvegarde si nécessaire.
5. **Validation** — ce qui fonctionne réellement, et ce qui n'a pas été testé.
6. **Retour arrière** — procédure pour retrouver l'état précédent.
7. **RETEX** — erreurs, décisions, statut, prochaine action.

La validation d'un diagnostic ne vaut pas validation de toute une architecture.

3. Lire l'état de la machine sans rien changer

Exemples d'observation sur une distribution Linux récente :

```
uname -a
cat /etc/os-release
systemctl --failed
```

```
lsblk -f
df -h
```

Contrôle : comprendre la sortie et confirmer le bon appareil avant toute intervention. Ne jamais publier un rapport brut de diagnostic sans examiner les noms de comptes, périphériques, montages et métadonnées qu'il contient.

4. Dépanner une application : l'exemple de la pile audio

Symptôme documenté : un navigateur charge les pages mais la vidéo reste bloquée ; un second navigateur fonctionne. La cause effective provenait de services audio utilisateur défaillants, et non d'une panne générale de réseau.

Vérifier avant d'agir :

```
systemctl --user is-active pipewire pipewire-pulse wireplumber
pactl info
```

Intervention ciblée, seulement si le diagnostic la justifie :

```
systemctl --user restart wireplumber pipewire pipewire-pulse
```

Validation : retester la lecture vidéo, la sortie audio et l'état des services. **Limite :** cette commande n'est pas une réparation universelle des problèmes de navigateur ; elle peut interrompre brièvement les flux audio en cours.

5. Fichiers, disques et montages

Objectif : savoir ce qui est monté, distinguer disque physique, partition, système de fichiers et point de montage.

```
lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINTS
findmnt
```

Précaution : ne pas formater, démonter ou réécrire une partition sur la seule base de son nom apparent. Conserver une sauvegarde séparée avant les changements de stockage.

6. Gestion de services

Le manuel distingue les services système des services de session utilisateur.

```
systemctl --failed
systemctl --user --failed
journalctl --user -b --priority=warning
```

RETEX utile : noter le service concerné, le message d'erreur observé, la mesure prise et le test post-correction. Ne pas interpréter « service actif » comme une preuve que toute la fonction métier est opérationnelle.

7. Réseau et accès distant — principes généraux

La documentation privée porte sur des systèmes reliés entre eux, mais la version publique ne fournit **aucune adresse, route, nom de machine réel ni configuration d'accès**. Les bonnes pratiques réutilisables sont : limiter les services exposés, appliquer le principe du moindre privilège, utiliser des mécanismes d'authentification adaptés et tester l'accès sur le canal voulu.

Ne jamais publier : fichiers de tunnel, clés, empreintes de clients, adresses, règles de transfert, inventaires de ports ou historiques d'accès. Les diagrammes publics doivent présenter des rôles génériques, pas une topologie reproductible.

8. Appareils mobiles comme terminaux

Une tablette peut être un **client léger** vers une session de travail persistante sur un ordinateur, plutôt que d'exécuter localement toutes les tâches lourdes.

Principe : interface mobile → connexion sécurisée → session terminal persistante → travail sur la machine principale. Séparer les actions de lecture des actions modifiant des fichiers ; ne pas exposer de commandes destructives derrière une interface vocale ou automatisée.

9. Sauvegardes : trois vérifications distinctes

Une copie de fichier ne suffit pas à prouver une restauration réussie. Le manuel distingue :

- **Copie effectuée** — les fichiers ont été produits à l'emplacement prévu.
- **Intégrité vérifiée** — les empreintes concordent.
- **Restauration / lecture vérifiée** — un test récupère réellement le contenu attendu.

Exemples génériques :

```
sha256sum fichier-exemple.zip
unzip -t fichier-exemple.zip
```

La restauration complète à blanc peut rester « à tester » même lorsque les deux premières étapes sont validées. Enregistrer explicitement cette limite.

10. Git : préparer sans écraser

Pour travailler sur une évolution de site ou de script :

```
git status --short
git diff --check
git log --oneline -5
```

Bon réflexe : comprendre les modifications avant commit, isoler les changements dans une branche lorsque c'est utile, vérifier les fichiers générés et ne pas publier de données privées dans l'historique. Une suppression Git ou un push est une opération de changement d'état : elle demande un examen spécifique.

11. Le RETEX comme passage de relais

Une fiche de fin de séance doit répondre sans ambiguïté à cinq questions : **qu’avons-nous fait ? qu’est-ce qui a marché ? qu’est-ce qui a échoué ? qu’avons-nous décidé ? que faire ensuite ?**

Mini-exemple :

```
Sujet : incident de lecture multimédia
Hypothèses testées : navigateur / réseau / audio utilisateur
Résultat validé : retour de la lecture après correction ciblée de l'audio
Limites : aucune généralisation à tous les incidents vidéo
Prochaine séance : surveiller les journaux en cas de récidive
```

Ce document peut être repris dans une autre conversation en étant **lu et vérifié**. Il ne constitue pas une mémoire automatique et parfaite du modèle.

12. Statuts de preuve

Statut	Signification
VALIDÉ	Fonction testée dans le périmètre annoncé
À TESTER	Proposition ou procédure non encore démontrée
ÉCHEC CONNU	Tentative documentée, non conservée comme solution
CORRIGÉ	Anomalie résolue et retestée sur un périmètre explicite
ARCHIVÉ	Version conservée ; cela n’implique pas qu’elle soit à jour

Une affirmation doit correspondre au test exact, pas au résultat espéré.

13. Ce que cette expérience montre

En moins d’un an d’utilisation de l’IA, le changement le plus précieux est l’apparition d’un **cycle d’apprentissage continu** : une question devient une procédure, une erreur devient une règle de contrôle, une solution devient une fiche et plusieurs fiches construisent un manuel. L’humain doit toujours comprendre, choisir, vérifier et conserver la maîtrise de l’environnement.

Périmètre de l’édition publique : démonstration méthodologique et exemples non sensibles. Aucun inventaire réel, aucun fichier de configuration personnel, aucun lien privé, aucune adresse réseau, aucun secret, aucun journal brut de session. Le manuel source, évolutif, est conservé hors du dépôt public.

© Laurent B. 2026. Diffusion de lecture ; aucune autorisation implicite de réutilisation des autres œuvres du projet.